qsBye

折腾笔记[24]-使用rust基于光流法对比图像

摘要

使用rust推理onnx模型获取两张图片的光流信息, 比较图片之间的差异并使用色轮绘制差异部分.
Use Rust to infer an ONNX model to obtain the optical flow information between two images, compare the differences between the images, and draw the different parts using a color wheel.

关键词

rust;ort;optical flow;neuflow;

关键信息

项目地址:
[https://github.com/ByeIO/bye.orbslam3.rs/blob/dev1/crates/seekslam_examples/examples/ort_optics.rs]
配置文件:

```
[workspace.package]
version = "0.0.1"
edition = "2024"
[workspace.dependencies]
# 错误处理
anyhow = "1.0.97"
# 时间格式化
chrono = "0.4.40"
# Model Context Protocol协议定义
rust-mcp-schema = { version = "0.2.2", path = "./static/rust-mcp-schema" }
# Model Context Protocol开发工具
# rust-mcp-sdk = { version = "0.1.2", path = "./static/rust-mcp-sdk/crates/rust-mcp-sdk" }
rust-mcp-sdk = { version = "0.1.2" }
# 多线程框架
tokio = { version = "1.44.1", features = ["full"] }
# ros2接口
ros2-interfaces-humble = "0.0.1"
# 日志后端
log = { version = "0.4.27", features = ["std"] }
# prototxt文件读取
protokit = "0.2.0"
# protobuf处理
prost = "0.13.5"
prost-build = { version = "0.13.5", features = ["cleanup-markdown"] }
# onnx运行时
wonnx = { version = "0.5.1", path = "./static/wonnx/wonnx" }
# 线性代数
nalgebra = { version = "0.33.2", features = ["rand"] }
ndarray = { version = "0.16.1", path = "./static/ndarray" }
# 随机数
rand = "0.9.0"
rand_distr = "0.5.1"
# 图像处理
image = "0.25.6"
imageproc = "0.25.0"
# 图优化
factrs = "0.2.0"
# wasm运行时
wasmtime-cli = { version = "31.0.0", path="./static/wasmtime-cli-31.0.0" }
# tract运行时
tract-onnx = { version = "0.21.11", features = ["getrandom-js"], path = "./static/tract-onnx"
# javascript/ESM运行时
deno_cli = { version = "2.2.8", path = "./static/deno" }
# 临时文件
tempfile = "3.19.1"
# 嵌入文件
embed-file = "0.2.0"
# onnxruntime(C绑定), 默认特性
ort = { version = "2.0.0-rc.9", path = "./static/ort" }
# YOLOv11
yolo-rs = "0.1.2"
# 可克隆的迭代器
itertools = "0.14.0"
# 克隆工具
clone_dyn_types = "0.29.0"

[patch.crates-io]
# 替换依赖文件
ndarray = { version = "0.16.1", path = "./static/ndarray" }
```

原理简述

光流概念

[https://xinetzone.github.io/torch-book/ecosystem/tutorials/vision-transforms/optical-flow.html]
光流是预测两幅图像之间运动的任务，通常是视频中连续两帧之间的运动。光流模型接受两幅图像作为输入，并预测流场：流场指示第一幅图像中每个像素的位移，并将其映射到第二幅图像中的相应像素。流场是 (2, H, W) 维张量，其中第一个轴对应于预测的水平位移和垂直位移。需要注意的是，预测的流场是以“像素”为单位的，它们并没有根据图像的尺寸进行归一化。
在光流中，每个“方向”都将映射为特定的RGB颜色。模型认为颜色相近的像素正在朝相似方向移动。
inference：用训练好的模型对连续图像帧求光流。

光流推理模型重要概念

[https://zhuanlan.zhihu.com/p/716470435]

1. 光流可视化

[https://blog.csdn.net/qq_33757398/article/details/106332814]
光流可视化是将光流信息以直观的方式展示出来，便于理解图像序列中物体的运动情况。根据光流的密度，可以分为稠密光流可视化和稀疏光流可视化。此外，光流映射（warp）是将光流应用到图像上，生成运动后的图像。以下是详细解释：

稠密光流可视化

稠密光流可视化是将每一像素的光流信息都展示出来，通常使用颜色来表示光流的方向和大小。颜色编码通常基于一个“色轮”（color wheel），其中不同的颜色代表不同的运动方向，颜色的亮度或饱和度表示运动的大小。例如：

- **绿色**可能表示向右上方运动。
- **红色**可能表示向左下方运动。
- 颜色的深度（饱和度）表示运动的大小，颜色越深表示运动越快。

这种可视化方式可以清晰地展示整个图像的运动情况，但可能会因为信息过于密集而难以区分细节。

稀疏光流可视化

稀疏光流可视化是只展示部分像素的光流信息，通常通过采样来减少显示的光流点。例如，可以每隔几个像素点才显示一个光流向量。这种方式的优点是能够更清晰地展示光流的方向和大小，避免了稠密光流可视化中可能出现的信息过载问题。

在稀疏光流可视化中，通常会使用箭头来表示光流的方向和大小，箭头的长度和方向对应光流的大小和方向。

光流映射（warp）

光流映射是将光流信息应用到图像上，生成一张新的图像，这张新图像反映了原始图像在光流作用下的运动效果。具体来说，就是根据光流向量将原始图像中的每个像素移动到新的位置，然后通过插值等方法生成新的图像。

光流映射有两种常见的插值方式：

1. **最近邻插值（nearest）**：直接选择最近的像素值。
2. **双线性插值（bilinear）**：根据周围四个像素的值通过线性插值计算目标像素的值。

光流映射可以用于多种应用，例如视频稳定、运动估计、图像合成等。

总结

- **稠密光流可视化**：展示所有像素的光流信息，适合整体观察运动情况。
- **稀疏光流可视化**：展示部分像素的光流信息，适合观察细节。
- **光流映射**：将光流应用到图像上，生成运动后的图像，可用于多种实际应用。

2. 在多尺度下寻找匹配度的金字塔

多尺度匹配度金字塔是光流估计中常用的一种技术，它通过在不同分辨率层次上寻找图像之间的匹配度来提高光流估计的精度。这种方法首先将输入图像构建为一个金字塔结构，每一层代表一个不同的分辨率。然后，从最粗糙的层级开始，逐层细化光流估计。在每一层，模型会计算特征匹配的成本体积，并利用这些信息来预测该层级的光流。随后，这些光流估计会被上采样到下一层，并与下一层的特征匹配信息结合，进一步细化光流估计。这种方法可以有效处理大位移运动，并减少计算量。

3. 常用数据集

光流估计的常用数据集包括：

- **FlyingChairs**：包含22872个训练图像对，通过随机仿射变换生成第二帧图像。
- **FlyingThings3D**：包含22k训练图像对，拥有3D运动和光照渲染效果。
- **Sintel**：包含Clean和Final两个子类数据集，共有1041个训练图像对和552个测试图像对。
- **KITTI**：包含200个训练图像对和200个测试图像对，主要关注真实道路交通场景。
- **HD1K**：包含1047个训练图像对，主要在交通场景中捕获。

这些数据集为光流估计模型提供了丰富的训练和测试材料，涵盖了从合成数据到真实场景的多种情况。

4. 双线性插值

双线性插值是一种在图像处理中常用的插值方法，用于在光流估计中将低分辨率的光流图上采样到高分辨率。这种方法通过考虑四个最近邻像素的加权平均来计算新像素的值。在光流估计中，双线性插值可以用来将粗略的光流场细化到原始图像的分辨率。然而，这种方法可能会在运动边界区域产生模糊，因此一些先进的方法如RAFT采用了凸上采样技术来改善这一问题。

5. GRU公式

GRU（Gated Recurrent Unit）是一种循环神经网络的变体，它通过引入更新门和重置门来控制信息的流动。GRU的公式如下：

- 更新门 z_t：

$$z_t = \sigma(W^z x_t + U^z h_{t-1})$$

- 重置门 r_t：

$$r_t = \sigma(W^r x_t + U^r h_{t-1})$$

- 候选隐藏状态 $\tilde{h}_t$：

$$\tilde{h}_t = \tanh(W^h x_t + U^h (h_{t-1} \odot r_t))$$

- 最终隐藏状态 h_t：

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

其中，σ 是sigmoid激活函数，W 和 U 是权重矩阵，x_t 是当前输入，h_{t-1} 是前一时刻的隐藏状态。GRU通过这些门控制信息的保留和更新，从而在序列数据处理中表现出色。

6. LSTM公式

LSTM（Long Short-Term Memory）是另一种循环神经网络的变体，它通过引入三个门（输入门、遗忘门、输出门）来控制信息的流动。LSTM的公式如下：

- 遗忘门 f_t：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- 输入门 i_t：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- 候选细胞状态 $\tilde{C}_t$：

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- 最终细胞状态 C_t：

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- 输出门 o_t：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- 最终隐藏状态 h_t：

$$h_t = o_t \cdot \tanh(C_t)$$

其中，W 是权重矩阵，b 是偏置项，x_t 是当前输入，h_{t-1} 是前一时刻的隐藏状态，C_{t-1} 是前一时刻的细胞状态。LSTM通过这些门控制信息的长期和短期流动，从而在处理长序列数据时避免梯度消失问题。

7. 上采样

上采样是光流估计中将低分辨率光流图恢复到原始分辨率的过程。常用的方法包括双线性插值、双三次插值和最近邻插值等。在深度学习中，还可以使用转置卷积（也称为反卷积）来实现上采样。转置卷积通过学习上采样的权重来恢复图像的高分辨率细节。此外，一些先进的方法如RAFT采用了凸上采样技术，它通过学习上采样权重来指导光流的上采样过程，从而在恢复高分辨率光流图时保留运动边缘的清晰度。

总结来说，光流推理模型的重要概念涵盖了从特征提取、多尺度匹配、数据集、插值方法到循环神经网络的多个方面。这些概念不仅为理解光流估计提供了基础，也为开发和改进光流推理模型提供了关键的技术支持。

光流跟踪过程

知乎 @奔跑的小伟
光流跟踪过程：
1） 对一个连续的视频图像帧序列进行处理；
2） 针对每一帧图像检测前景目标；
3）如果某一帧出现了前景目标，找到其具有代表性的特征点（一般利用角点来做特征点）；
4）对于之后的两个相邻视频帧，寻找上一帧中出现的特征点在当前帧中的最佳位置，从而得到前景目标在当前帧中的位置信息；
5）如此迭代进行，便可实现目标的跟踪；

光流推理模型综述

[https://zhuanlan.zhihu.com/p/623489787]
[https://github.com/cuixing158/Awesome-CV-MasterHub/tree/main]

1. RAFT
2. AutoFlow
3. NeoFlow(v1,v2)

1. RAFT（Recurrent All-Pairs Field Transforms）

- **简介**：RAFT是一种基于深度学习的光流估计模型，由Princeton Vision Lab提出。它通过循环的方式逐步细化光流估计，具有较高的精度和鲁棒性。
- **核心思想**：
    - **特征提取**：使用卷积神经网络提取两帧图像的特征图。
    - **循环细化**：通过循环网络（GRU）逐步更新光流估计，每次迭代都会考虑所有像素对之间的关系。
    - **双向光流估计**：同时估计正向和反向光流，通过一致性检查提高精度。
- **优点**：
    - 高精度：在多个基准数据集上表现出色。
    - 鲁棒性：对光照变化、遮挡等复杂场景有较好的适应性。
- **缺点**：
    - 计算复杂度较高：由于循环细化过程，推理速度相对较慢。
- **适用场景**：适用于对精度要求较高的光流估计任务，如视频分析、运动估计等。

2. AutoFlow

- **简介**：AutoFlow是一种自动化的光流估计框架，旨在通过自动化设计和优化光流网络来提高性能。
- **核心思想**：
    - **自动化网络设计**：使用神经架构搜索（NAS）技术自动设计光流网络结构。
    - **多尺度特征融合**：通过多尺度特征融合提高光流估计的精度。
    - **轻量化设计**：在保证精度的同时，优化网络结构以提高推理速度。
- **优点**：
    - 自动化设计：减少了人工设计网络的复杂性。
    - 高效性：通过优化网络结构，提高了推理速度。
- **缺点**：
    - 依赖NAS技术：需要大量的计算资源进行网络搜索。
- **适用场景**：适用于需要高效光流估计的实时应用，如自动驾驶、视频流处理等。

3. NeoFlow（v1和v2）

- **NeoFlow v1**：
    - **简介**：NeoFlow v1是一种基于深度学习的光流估计模型，旨在提高光流估计的速度和精度。
    - **核心思想**：
        - **特征提取**：使用卷积神经网络提取图像特征。
        - **光流估计**：通过卷积操作估计光流，结合多尺度特征融合提高精度。
    - **优点**：
        - 较高的精度：在多个基准数据集上表现出色。
        - 较快的推理速度：相比传统方法有显著提升。
    - **缺点**：
        - 对复杂场景的适应性有限：在光照变化、遮挡等复杂场景下表现不如RAFT。
    - **适用场景**：适用于一般光流估计任务，如视频分析、运动估计等。
- **NeoFlow v2**：
    - **简介**：NeoFlow v2是NeoFlow v1的改进版本，进一步优化了网络结构和算法，提高了性能。
    - **核心思想**：
        - **改进的特征提取**：优化了特征提取网络，提高了特征的表达能力。
        - **双向光流估计**：引入双向光流估计，通过一致性检查提高精度。
        - **循环细化**：借鉴RAFT的思想，通过循环细化逐步优化光流估计。
    - **优点**：
        - 更高的精度：在复杂场景下表现更好。
        - 较快的推理速度：相比RAFT有显著提升。
    - **缺点**：
        - 网络结构复杂：需要更多的计算资源进行训练。
    - **适用场景**：适用于对精度和速度都有较高要求的光流估计任务，如自动驾驶、视频分析等。

总结

- **RAFT**：精度高，鲁棒性强，但计算复杂度高，适合对精度要求较高的场景。
- **AutoFlow**：自动化设计，高效性好，适合实时应用。
- **NeoFlow v1**：速度快，精度高，适合一般光流估计任务。
- **NeoFlow v2**：精度和速度兼顾，适合复杂场景下的光流估计任务。

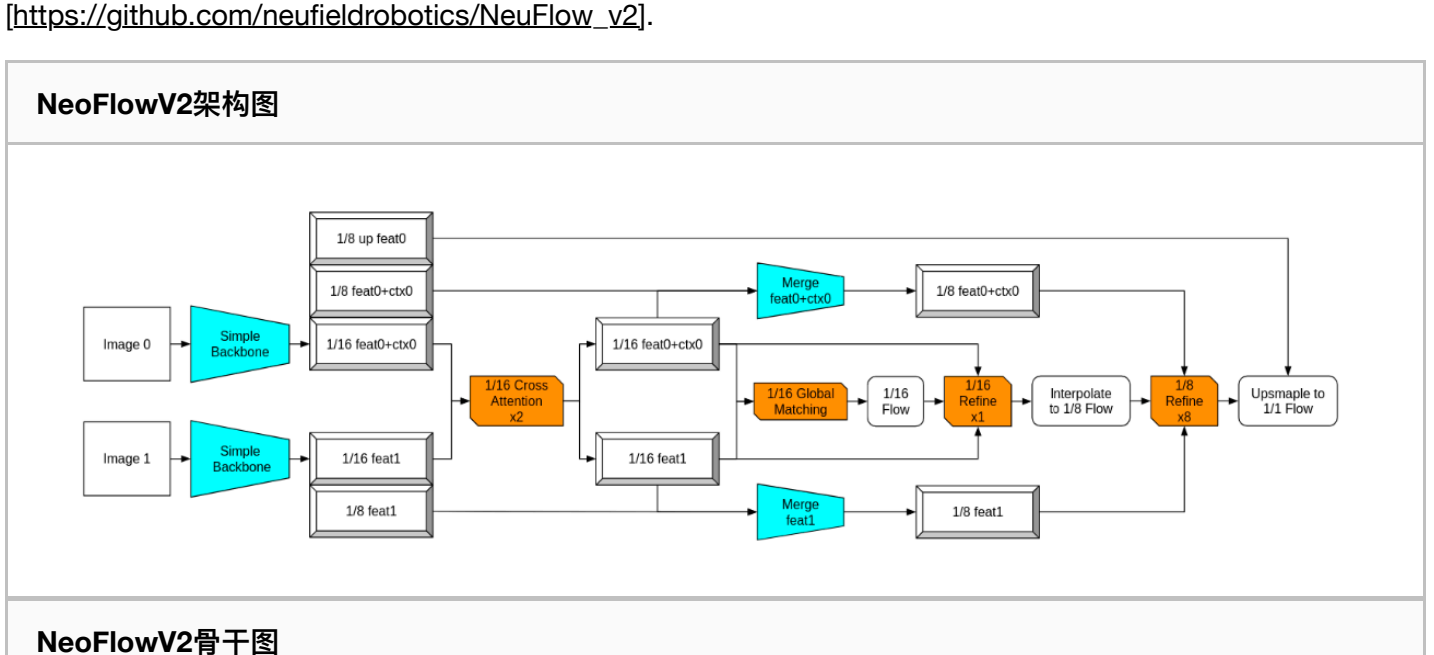
这些模型各有优缺点，选择时需要根据具体的应用场景和需求进行权衡。

NeoFlowV2数学原理简介

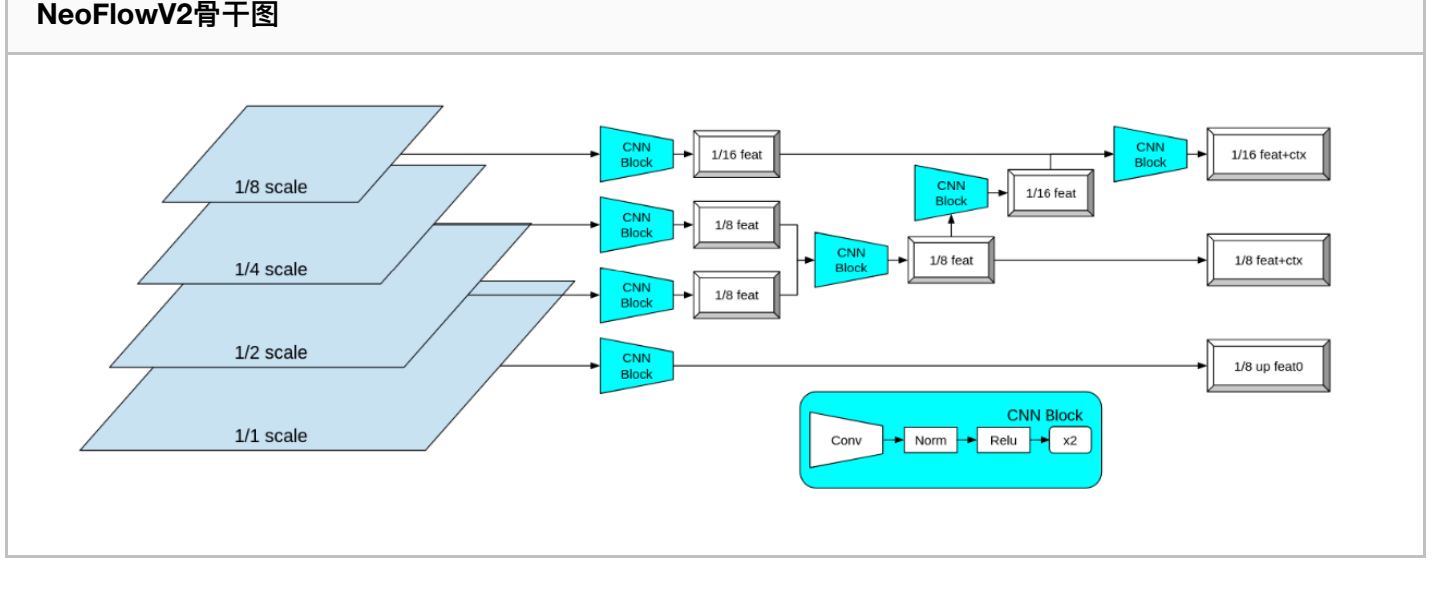
实时高精度光流估计对于各种实际应用场景至关重要。尽管最近基于学习的光流方法已经实现了高精度，但它们通常伴随着巨大的计算成本。在本文中，我们提出了一种高效的光流方法，能够在降低计算需求的同时保持高精度。在此基础上，我们引入了新的组件，包括一个更轻量级的主干网络和一个快速细化模块。这两个模块在保持计算需求较低的同时，提供了接近最先进的精度。与其他最先进的方法相比，我们的模型在保持合成数据和真实数据的可比性能的同时，实现了10倍到70倍的速度提升。它能够在Jetson Orin Nano上以超过20 FPS的速度运行512x384分辨率的图像。完整的训练和评估代码可在https://github.com/neufieldrobotics/NeuFlow_v2上找到。

Real-time high-accuracy optical flow estimation is crucial for various real-world applications. While recent learning-based optical flow methods have achieved high accu- racy, they often come with significant computational costs. In this paper, we propose a highly efficient optical flow method that balances high accuracy with reduced computational demands. Building upon NeuFlow v1, we introduce new components including a much more light-weight backbone and a fast refinement module. Both these modules help in keeping the computational demands light while providing close to state of the art accuracy. Compares to other state of the art methods, our model achieves a 10x-70x speedup while maintaining comparable performance on both synthetic and real-world data. It is capable of running at over 20 FPS on 512x384 resolution images on a Jetson Orin Nano. The full training and evaluation code is available at [https://github.com/neufieldrobotics/NeuFlow_v2].

NeoFlowV2架构图



NeoFlowV2骨干图



一、简单主干网络（Simple Backbone）

NeuFlow v2使用一个简单的CNN主干网络来提取多尺度图像的低级特征。主干网络从1/2、1/4和1/8尺度的图像中提取特征，使用由卷积层、归一化层和ReLU激活层组成的CNN块来提取特征，并将这些特征合并和调整大小到所需的输出尺度，即1/16尺度的特征和上下文，以及1/8尺度的特征和上下文。具体来说，对于输入图像I，主干网络的输出可以表示为：

$$F_{1/8}, F_{1/16}, C_{1/8}, C_{1/16} = \mathrm{Backbone}(I)$$

其中，$F_{1/8}$和$F_{1/16}$是用于相关性计算的特征，$C_{1/8}$和$C_{1/16}$是用于光流细化的上下文。

二、交叉注意力和全局匹配（Cross-Attention and Global Matching）

交叉注意力用于在全局范围内交换图像之间的信息，增强匹配特征的区分度，减少未匹配特征的相似度。全局匹配则用于在全局范围内寻找对应特征，使模型能够处理大像素位移，例如在快速移动的相机情况下。交叉注意力和全局匹配的操作可以表示为：

$$\begin{aligned} F'_{1/16} &= \mathrm{CrossAttention}(F_{1/16}, F_{1/16}) \\ F''_{1/16} &= \mathrm{GlobalMatching}(F'_{1/16}) \end{aligned}$$

其中，$F'_{1/16}$是经过交叉注意力增强后的特征，$F''_{1/16}$是经过全局匹配后的特征。

三、简单RNN细化模块（Simple RNN Refinement）

首先计算附近9×9邻域内的相关性，并使用估计的光流对相关性进行扭曲。然后将扭曲的相关性、上下文特征、估计的光流和前一隐藏状态连接起来，通过八层简单的3×3卷积层后接ReLU激活函数来输出细化后的光流和更新后的隐藏状态。简单RNN细化模块的操作可以表示为：

$$\begin{aligned} C &= \mathrm{ComputeCorrelation}(F''_{1/16}, F''_{1/16}) \\ C_w &= \mathrm{Warp}(C, F_{\mathrm{flow}}) \\ h_{t+1}, F^{t+1}_{\mathrm{flow}} &= \mathrm{SimpleRNN}(C_w, C_{1/16}, F^t_{\mathrm{flow}}, h_t) \end{aligned}$$

其中，C是计算的相关性，C_w是扭曲后的相关性，h_t是前一隐藏状态，F^t_{flow}是当前估计的光流，h_{t+1}和F^{t+1}_{flow}分别是更新后的隐藏状态和细化后的光流。

四、多尺度特征/上下文合并（Multi-Scale Feature/Context Merge）

为了将全局特征/上下文与局部特征/上下文合并，确保1/8尺度的特征/上下文包含全局和局部信息，使用简单的CNN块来合并1/16尺度的全局特征/上下文与1/8尺度的局部特征/上下文。合并操作可以表示为：

$$F^{\mathrm{merged}}_{1/8}, C^{\mathrm{merged}}_{1/8} = \mathrm{Merge}(F_{1/8}, F^{\mathrm{interpolated}}_{1/16}, C_{1/8}, C^{\mathrm{interpolated}}_{1/16})$$

其中，$F^{\mathrm{merged}}_{1/8}$和$C^{\mathrm{merged}}_{1/8}$是合并后的1/8尺度的特征和上下文。

五、整体流程

NeuFlow v2的整体流程可以总结为：

$$\begin{aligned} F_{1/8}, F_{1/16}, C_{1/8}, C_{1/16} &= \mathrm{Backbone}(I) \\ F'_{1/16} &= \mathrm{CrossAttention}(F_{1/16}, F_1 \\ F''_{1/16} &= \mathrm{GlobalMatching}(F'_{1/16}) \\ C &= \mathrm{ComputeCorrelation}(F''_{1/} \\ C_w &= \mathrm{Warp}(C, F^0_{\mathrm{flow}}) \\ h_1, F^1_{\mathrm{flow}} &= \mathrm{SimpleRNN}(C_w, C_{1/16}, F \\ F^{\mathrm{merged}}_{1/8}, C^{\mathrm{merged}}_{1/8} &= \mathrm{Merge}(F_{1/8}, F^{\mathrm{interpolated}}_{1/16}, \\ \text{Repeat SimpleRNN refinement for 8 iterations on } F^{\mathrm{merged}}_{1/8} \text{ and } C^{\mathrm{merged}}_{1/8} & \end{aligned}$$

最终，通过凸上采样模块将细化后的1/8尺度光流上采样到全分辨率，得到最终的光流估计结果。

实现

1. 准备onnx模型

[https://github.com/ibaiGorordo/ONNX-NeuFlowV2-Optical-Flow]
[https://github.com/ibaiGorordo/ONNX-NeuFlowV2-Optical-Flow/releases/download/0.1.0/neuflow_mixed.onnx]
[https://github.com/ibaiGorordo/ONNX-NeuFlowV2-Optical-Flow/releases/download/0.1.0/neuflow_sintel.onnx]
[https://github.com/ibaiGorordo/ONNX-NeuFlowV2-Optical-Flow/releases/download/0.1.0/neuflow_things.onnx]

2. 核心代码

Python版本:

```python
# 使用optics模型预测光流
# 模型：../../../assets/neuflow_v2/neoflow_things.onnx
# 输入:input1
# name: input1
# tensor: float32[1,3,432,768]
# input2
# name: input2
# tensor: float32[1,3,432,768]
# 输出:output
# name: output
# tensor: float32[1,2,432,768]

import cv2
import numpy as np
import onnxruntime
import os
import requests
from tqdm import tqdm

# 可用的模型列表
available_models = ["neuflow_mixed", "neuflow_sintel", "neuflow_things"]

# 检查模型文件是否存在，若不存在则下载
def check_model(model_path: str):
    if os.path.exists(model_path):
        print("模型文件存在!")
        return
    # 从路径中提取模型名称
    model_name = os.path.basename(model_path).split('.')[0]
    if model_name not in available_models:
        raise ValueError(f"无效的模型名称：{model_name}")
        exit(0)

# 绘制光流图
def draw_flow(flow, image, boxes=None):
    # 将光流转换为彩色图像
    flow_img = flow_to_image(flow, 35)
    # 从RGB颜色空间转换为BGR颜色空间
    flow_img = cv2.cvtColor(flow_img, cv2.COLOR_RGB2BGR)
    # 合并原始图像和光流图像
    combined = cv2.addWeighted(image, 0.5, flow_img, 0.6, 0)
    if boxes is not None:
        # 创建白色背景图像
        white_background = np.ones((image.shape[0], image.shape[1], 3), dtype=np.uint8) * 255
        # 合并原始图像和白色背景
        new_image = cv2.addWeighted(image, 0.7, white_background, 0.4, 0)
        for box in boxes:
            x1, y1, x2, y2 = box.astype(int)
            # 将合并后的图像部分替换到新图像中
            new_image[y1:y2, x1:x2] = combined[y1:y2, x1:x2]
        combined = new_image
    return combined

# 生成颜色轮
def make_color_wheel():
    """
    根据Middlebury颜色编码生成颜色轮
    :return: 颜色轮
    """
    RY = 15
    YG = 6
    GC = 4
    CB = 11
    BM = 13
    MR = 6
    ncols = RY + YG + GC + CB + BM + MR
    colorwheel = np.zeros([ncols, 3])
    col = 0
    # RY段
    colorwheel[0:RY, 0] = 255
    colorwheel[0:RY, 1] = np.transpose(np.floor(255 * np.arange(0, RY) / RY))
    col += RY
    # YG段
    colorwheel[col:col + YG, 0] = 255 - np.transpose(np.floor(255 * np.arange(0, YG) / YG))
    colorwheel[col:col + YG, 1] = 255
    col += YG
    # GC段
    colorwheel[col:col + GC, 1] = 255
    colorwheel[col:col + GC, 2] = np.transpose(np.floor(255 * np.arange(0, GC) / GC))
    col += GC
    # CB段
    colorwheel[col:col + CB, 1] = 255 - np.transpose(np.floor(255 * np.arange(0, CB) / CB))
    colorwheel[col:col + CB, 2] = 255
    col += CB
    # BM段
    colorwheel[col:col + BM, 2] = 255
    colorwheel[col:col + BM, 0] = np.transpose(np.floor(255 * np.arange(0, BM) / BM))
    col += BM
    # MR段
    colorwheel[col:col + MR, 2] = 255 - np.transpose(np.floor(255 * np.arange(0, MR) / MR))
    colorwheel[col:col + MR, 0] = 255
    return colorwheel

# 计算光流颜色图
colorwheel = make_color_wheel()
def compute_color(u, v):
    """
    计算光流颜色图
    :param u: 光流水平分量图
    :param v: 光流垂直分量图
    :return: 颜色编码的光流图
    """
    [h, w] = u.shape
    img = np.zeros([h, w, 3])
    # 找出NaN值的索引
    nanIdx = np.isnan(u) | np.isnan(v)
    u[nanIdx] = 0
    v[nanIdx] = 0
    ncols = np.size(colorwheel, 0)
    # 计算光流的幅值
    rad = np.sqrt(u ** 2 + v ** 2)
    # 计算光流的角度
    a = np.arctan2(-v, -u) / np.pi
    fk = (a + 1) / 2 * (ncols - 1) + 1
    k0 = np.floor(fk).astype(int)
    k1 = k0 + 1
    k1[k1 == ncols + 1] = 1
    f = fk - k0
    for i in range(0, np.size(colorwheel, 1)):
        tmp = colorwheel[:, i]
        col0 = tmp[k0 - 1] / 255
        col1 = tmp[k1 - 1] / 255
        col = (1 - f) * col0 + f * col1
        idx = rad <= 1
        col[idx] = 1 - rad[idx] * (1 - col[idx])
        notidx = np.logical_not(idx)
        col[notidx] *= 0.75
        img[:, :, i] = np.uint8(np.floor(255 * col * (1 - nanIdx)))
    return img

# 将光流转换为Middlebury颜色编码图像
def flow_to_image(flow, maxrad=None):
    """
    将光流转换为Middlebury颜色编码图像
    :param flow: 光流图
    :return: Middlebury颜色编码的光流图像
    """
    u = flow[:, :, 0]
    v = flow[:, :, 1]
    rad = np.sqrt(u ** 2 + v ** 2)
    if maxrad is None:
        maxrad = max(-1, np.max(rad))
    eps = np.finfo(float).eps
    u = np.clip(u, -maxrad + 5, maxrad - 5)
    v = np.clip(v, -maxrad + 5, maxrad - 5)
    u = u / (maxrad + eps)
    v = v / (maxrad + eps)
    img = compute_color(u, v)
    return np.uint8(img)

# 光流估计类
class NeuFlowV2:
    def __init__(self, path: str):
        # 检查模型文件
        check_model(path)
        # 初始化ONNX运行时会话
        self.session = onnxruntime.InferenceSession(path, providers=onnxruntime.get_available_
        # 获取输入信息
        self.get_input_details()
        # 获取输出信息
        self.get_output_details()

    def __call__(self, img_prev: np.ndarray, img_now: np.ndarray) -> np.ndarray:
        return self.estimate_flow(img_prev, img_now)

    # 估计光流
    def estimate_flow(self, img_prev: np.ndarray, img_now: np.ndarray) -> np.ndarray:
        # 准备输入张量
        input_tensors = self.prepare_inputs(img_prev, img_now)
        # 进行推理
        outputs = self.inference(input_tensors)
        return self.process_output(outputs[0])

    # 准备输入张量
    def prepare_inputs(self, img_prev: np.ndarray, img_now: np.ndarray) -> tuple[np.ndarray, 
        self.img_height, self.img_width = img_now.shape[:2]
        input_prev = self.prepare_input(img_prev)
        input_now = self.prepare_input(img_now)
        return input_prev, input_now

    # 准备单张输入图像
    def prepare_input(self, img: np.ndarray) -> np.ndarray:
        # 调整图像大小
        input_img = cv2.resize(img, (self.input_width, self.input_height))
        # 归一化
        input_img = input_img / 255.0
        input_img = input_img.transpose(2, 0, 1)
        input_tensor = input_img[np.newaxis, :, :, :].astype(np.float32)
        return input_tensor

    # 进行推理
    def inference(self, input_tensors: tuple[np.ndarray, np.ndarray]) -> np.ndarray:
        start = time.perf_counter()
        outputs = self.session.run(self.output_names, {self.input_names[0]: input_tensors[0],
                                                       self.input_names[1]: input_tensors[1]}
        print(f"推理时间：{(time.perf_counter() - start) * 1000:.2f} ms")
        return outputs

    # 处理输出结果
    def process_output(self, output) -> np.ndarray:
        flow = output.squeeze().transpose(1, 2, 0)
        return cv2.resize(flow, (self.img_width, self.img_height))

    # 获取输入信息
    def get_input_details(self):
        model_inputs = self.session.get_inputs()
        self.input_names = [model_inputs[i].name for i in range(len(model_inputs))]
        input_shape = model_inputs[0].shape
        self.input_height = input_shape[2]
        self.input_width = input_shape[3]

    # 获取输出信息
    def get_output_details(self):
        model_outputs = self.session.get_outputs()
        self.output_names = [model_outputs[i].name for i in range(len(model_outputs))]

if __name__ == '__main__':
    import time
    # 模型文件路径
    model_path = "../../../assets/ailia-models/neuflow_v2/neuflow_sintel.onnx"
    print("模型文件路径{}", model_path)
    # 初始化模型
    estimator = NeuFlowV2(model_path)
    # 加载第一张图片
    img1 = cv2.imread("../assets/frame_0016.png")
    # 加载第二张图片
    img2 = cv2.imread("../assets/frame_0025.png")
    # 估计光流
    flow = estimator(img1, img2)
    # 绘制光流图
    flow_img = draw_flow(flow, img1)
    # 创建结果文件夹
    os.makedirs("../result", exist_ok=True)
    # 保存结果图片
    cv2.imwrite("../result/optics_onnx.png", flow_img)
    print("光流预测结果已保存到 ../result/optics_onnx.png")
```

Rust版本:

```rust
#![allow(unused)]

//! 使用optics模型预测光流
//! 模型：../../../assets/neuflow_v2/neoflow_things.onnx
//! 输入:input1
//! name: input1
//! tensor: float32[1,3,432,768]
//! input2
//! name: input2
//! tensor: float32[1,3,432,768]
//! 输出:output
//! name: output
//! tensor: float32[1,2,432,768]

// 导入标准库路径处理模块
use std::path::Path;
[illegible]
```

公告

昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0

< 2025年4月 >

日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

搜索

常用链接

- 我的随笔
- 我的评论
- 我的参与
- 最新评论
- 我的标签

我的标签

- Rust(19)
- SLAM(6)
- 线性代数(2)
- LLM(1)

随笔分类

- 51单片机(10)
- Android开发(18)
- bouffalo单片机(1)
- esp32单片机(24)
- GNU Radio(4)
- LLM精彩问答(7)
- MATLAB(6)
- RaspberryPi单片机(6)
- SCILAB(3)
- stm32单片机(16)
- TI单片机(3)
- WebAssembly(3)
- 翻译(6)
- 高云FPGA(3)
- 公众号(2)
- 鸿蒙开发(3)
- 机器人(13)
- 前端开发(12)
- 嵌入式Linux(3)
- 人工智能(4)
- 运维相关(16)
- 折腾笔记(26)

随笔档案

- 2025年4月(8)
- 2025年3月(4)
- 2025年2月(6)
- 2025年1月(11)
- 2024年12月(1)
- 2024年10月(1)
- 2024年9月(4)
- 2024年8月(2)
- 2024年6月(3)
- 2024年5月(9)
- 2024年4月(3)
- 2024年3月(10)
- 2024年2月(8)
- 2024年1月(9)
- 2023年12月(4)
- 2023年11月(6)
- 2023年10月(4)
- 2023年9月(4)
- 2023年8月(10)
- 2023年7月(16)
- 2023年6月(2)
- 2023年5月(2)
- 2023年4月(3)
- 2023年3月(2)
- 2023年2月(7)
- 2023年1月(17)
- 2022年12月(11)
- 2022年11月(14)
- 2022年10月(8)
- 2022年9月(17)
- 更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11766)
2. esp32笔记[5]-基于I2S协议实现音频播放(9707)
3. 谈谈NAS的硬盘SATA背板(6823)
4. 三星平板的笔记模板(比例:16:10)(4358)
5. 电犀牛R66s软路由体验(4314)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

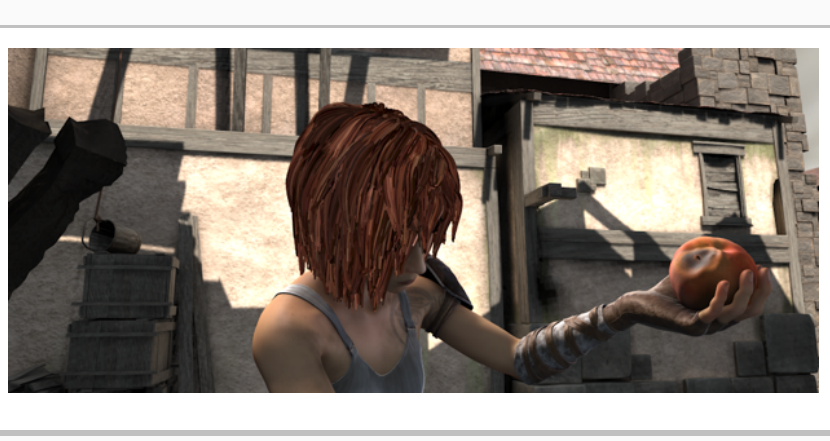
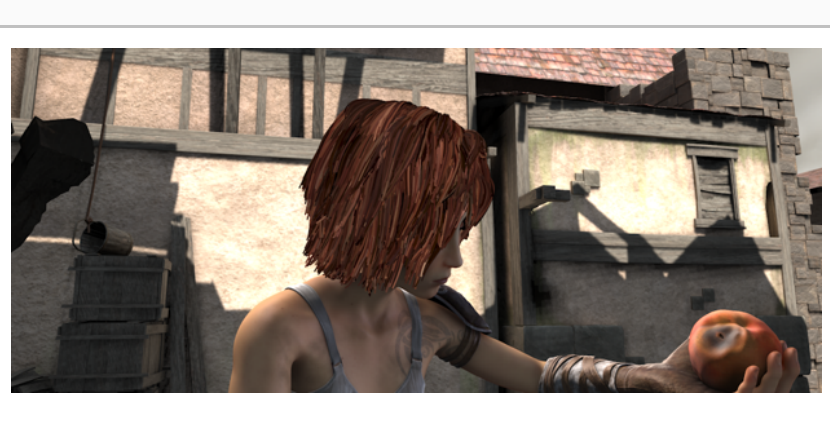
1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[1]-blink闪灯(1)
4. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

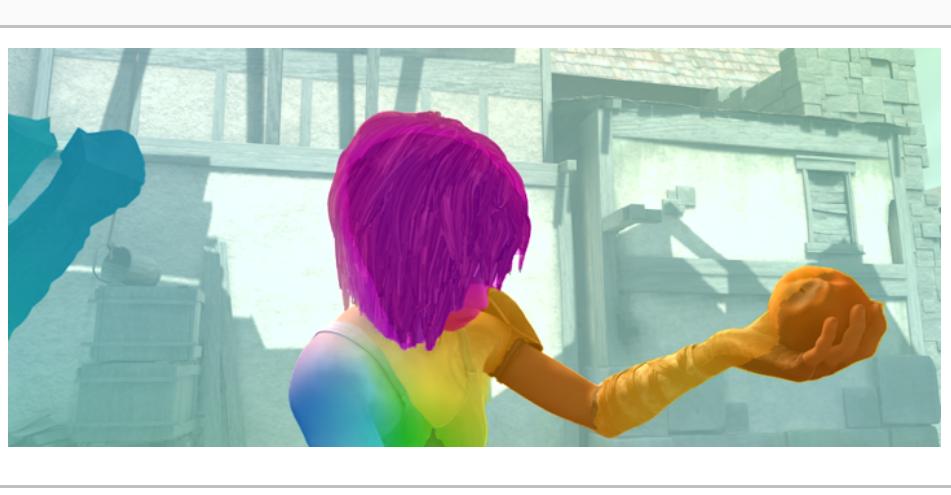
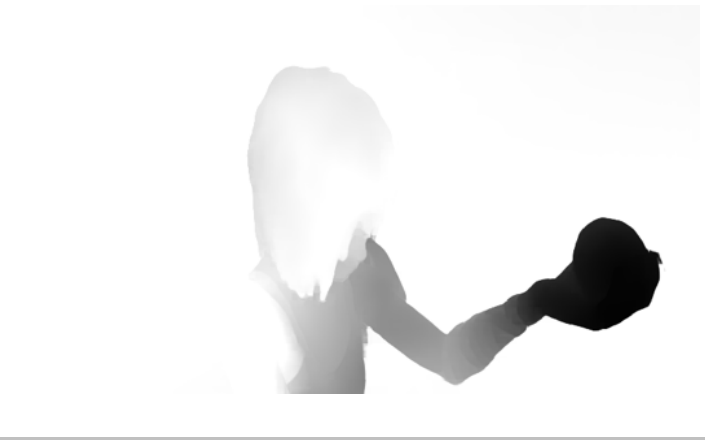
最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢?
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

```rust
use ort::value::Tensor;

// 图像处理相关模块
use image::{GenericImageView, ImageBuffer, Rgb, RgbImage, DynamicImage};

// 错误处理模块
use anyhow::{anyhow, Result};

// 线性代数库
use ndarray::{
    Array, ArrayD, ArrayViewD,
    Dim, Ix1, Ix2, Ix3, Ix4,
    IxDyn, stack, Axis, IxDynImpl,
    ArrayBase, OwnedRepr, IntoDimension,
    ArrayView,
};
use ndarray::s;
// use ndarray_linalg::error::LinalgError;

// 随机数
use rand::rng;

// 随机分布
use rand_distr::{Normal, Distribution};

// 可克隆的迭代器
use itertools::Itertools;

// 克隆插件
use clone_dyn_types::CloneDyn;

// 可用的模型列表
const AVAILABLE_MODELS: [&str; 3] = ["neuflow_mixed", "neuflow_sintel", "neuflow_things"];

// 检查模型文件是否存在，若不存在则处理（这里原Python代码没有实际下载逻辑，直接退出）
fn check_model(model_path: &str) -> Result<()> {
    if Path::new(model_path).exists() {
        println!("模型文件存在!");
        Ok(())
    } else {
        // 从路径中提取模型名称
        let model_name = Path::new(model_path).file_stem().ok_or_else(|| anyhow!("无法获取模型文
        if !AVAILABLE_MODELS.contains(&model_name) {
            return Err(anyhow!("无效的模型名称: {}", model_name));
        }
        // 原Python代码中下载部分被注释，这里直接退出
        std::process::exit(0);
    }
}

// 生成颜色轮
fn make_color_wheel() -> ArrayD<u8> {
    let ry = 15;
    let yg = 6;
    let gc = 4;
    let cb = 11;
    let bm = 13;
    let mr = 6;
    let ncols = ry + yg + gc + cb + bm + mr;
    let mut colorwheel = Array::zeros((ncols, 3));

    let mut col = 0;
    // RY段
    colorwheel.slice_mut(s![col..col + ry, 0]).fill(255);
    for (i, val) in (0..ry).enumerate() {
        colorwheel[[col + i, 1]] = ((255.0 * val as f32 / ry as f32) as u8);
    }
    col += ry;
    // YG段
    for (i, val) in (0..yg).enumerate() {
        colorwheel[[col + i, 0]] = (255 - (255.0 * val as f32 / yg as f32) as u8);
    }
    colorwheel.slice_mut(s![col..col + yg, 1]).fill(255);
    col += yg;
    // GC段
    colorwheel.slice_mut(s![col..col + gc, 1]).fill(255);
    for (i, val) in (0..gc).enumerate() {
        colorwheel[[col + i, 2]] = ((255.0 * val as f32 / gc as f32) as u8);
    }
    col += gc;
    // CB段
    for (i, val) in (0..cb).enumerate() {
        colorwheel[[col + i, 1]] = (255 - (255.0 * val as f32 / cb as f32) as u8);
    }
    colorwheel.slice_mut(s![col..col + cb, 2]).fill(255);
    col += cb;
    // BM段
    colorwheel.slice_mut(s![col..col + bm, 2]).fill(255);
    for (i, val) in (0..bm).enumerate() {
        colorwheel[[col + i, 0]] = ((255.0 * val as f32 / bm as f32) as u8);
    }
    col += bm;
    // MR段
    for (i, val) in (0..mr).enumerate() {
        colorwheel[[col + i, 2]] = (255 - (255.0 * val as f32 / mr as f32) as u8);
    }
    colorwheel.slice_mut(s![col..col + mr, 0]).fill(255);

    colorwheel.into_dyn()
}

// 计算光流颜色图
fn compute_color(u: &ArrayD<f32>, v: &ArrayD<f32>) -> ImageBuffer<Rgb<u8>, Vec<u8>> {
    // 获取输入数组的形状（高度和宽度）
    let (h, w) = {
        let shape = u.shape();
        (shape[0], shape[1])
    };
    // 创建一个与输入数组大小相同的图像缓冲区
    let mut img = ImageBuffer::new(w as u32, h as u32);

    // 初始化一个数组，用于标记存在 NaN 值的位置
    let mut nan_idx = Array::zeros((h, w));
    // 遍历 u 数组，标记 NaN 值的位置
    for (i, &val_u) in u.iter().enumerate() {
        let (y, x) = (i / w, i % w);
        if val_u.is_nan() {
            nan_idx[[y, x]] = 1.0;
        }
    }
    // 遍历 v 数组，标记 NaN 值的位置
    for (i, &val_v) in v.iter().enumerate() {
        let (y, x) = (i / w, i % w);
        if val_v.is_nan() {
            nan_idx[[y, x]] = 1.0;
        }
    }

    // 持久变量
    let color_wheel = make_color_wheel();

    // 获取颜色轮的列数
    let ncols = color_wheel.shape()[0];
    // 计算每个像素的径向大小
    let rad = Array::from_shape_fn((h, w), |(y, x)| {
        let u_val = u[[y, x]];
        let v_val = v[[y, x]];
        (u_val.powi(2) + v_val.powi(2)).sqrt()
    });
    // 计算每个像素的角度
    let a = Array::from_shape_fn((h, w), |(y, x)| {
        let u_val = u[[y, x]];
        let v_val = v[[y, x]];
        (-v_val).atan2(-u_val) / PI
    });
    // 将角度映射到颜色轮的索引
    let fk = a.mapv(|a_val| (a_val + 1.0) / 2.0 * (ncols as f32 - 1.0) + 1.0);
    // 获取颜色轮索引的整数部分
    let k0 = fk.mapv(|val| val.floor() as i32);
    // 获取颜色轮索引的上界整数部分
    let k1 = k0.mapv(|val| (val + 1).clamp(1, ncols as i32));
    // 计算颜色轮索引的小数部分
    let k0_float = k0.mapv(|val| val as f32);
    let f: Vec<f32> = fk.iter().zip(&k0_float).map(|(fk_val, k0_val)| fk_val - k0_val).collec

    // 遍历颜色通道
    for c in 0..3 {
        // 检查索引是否越界
        if c >= color_wheel.shape()[1] {
            panic!("Color channel index {} is out of bounds for color wheel with shape {:?}",
        }
        // 获取颜色轮对应通道的颜色值
        let colorwheel_col = color_wheel.slice(s![.., c]);
        // 获取颜色轮索引 k0 对应的颜色值
        let col0 = k0.mapv(|val| colorwheel_col[val as usize] as f32 / 255.0);
        // 获取颜色轮索引 k1 对应的颜色值
        let col1 = k1.mapv(|val| colorwheel_col[val.min((ncols-1).try_into().unwrap()) as usi
        // 插值计算最终的颜色值，返回可克隆的容器
        let col = f.iter().zip(&col0).zip(&col1).map(|((f_val, col0_val), col1_val)|
            (1.0 - f_val) * col0_val + f_val * col1_val
        ).collect_vec();

        // 根据径向大小调整颜色值
        let idx = rad.mapv(|rad_val| rad_val <= 1.0);

        let col_with_rad = col.into_iter().zip(&rad).map(|(col_val, rad_val)| {
            if *rad_val <= 1.0 {
                1.0 - rad_val * (1.0 - col_val)
            } else {
                col_val
            }
        });

        // 对超出径向范围的颜色值进行衰减
        let not_idx = idx.mapv(|val| !val);
        let col_final = col_with_rad.zip(&not_idx).map(|(col_val, not_idx_val)| if *not_idx_va

        // 将计算得到的颜色值赋值到图像缓冲区
        for (i, val) in col_final.enumerate() {
            let (y, x) = (i / w, i % w);
            let pixel_val = ((255.0 * val * (1.0 - nan_idx[[y, x]])) as u8).clamp(0, 255);
            img.put_pixel(x as u32, y as u32, Rgb([pixel_val, pixel_val, pixel_val]));
        }
    }

    // 返回生成的图像
    img
}

// 将光流转换为Middlebury颜色编码图像
fn flow_to_image(flow: &ArrayD<f32>) -> ImageBuffer<Rgb<u8>, Vec<u8>> {
    // 从光流数组中提取u分量（x方向运动），使用范围语法保持动态维度
    let u = flow.slice(s![0, 0, .., ..]);
    // 从光流数组中提取v分量（y方向运动），使用范围语法保持动态维度
    let v = flow.slice(s![0, 1, .., ..]);

    // 修正：使用map2替代zip+mapv组合
    let rad = u.mapv(|u_val| u_val.powi(2))
              .add(&v.mapv(|v_val| v_val.powi(2)))
              .mapv(|sum| sum.sqrt());

    // 找出所有像素中的最大光流幅度
    let maxrad = rad.iter().fold(f32::MIN, |acc, &val| acc.max(val));
    let eps = std::f32::EPSILON;

    // 将u分量裁剪到[-maxrad+5, maxrad-5]范围内
    let u_clipped = u.mapv(|val| val.clamp(-maxrad + 5.0, maxrad - 5.0));
    // 将v分量裁剪到[-maxrad+5, maxrad-5]范围内
    let v_clipped = v.mapv(|val| val.clamp(-maxrad + 5.0, maxrad - 5.0));

    // 将裁剪后的u分量归一化到[-1,1]范围
    let u_normalized = u_clipped.mapv(|val| val / (maxrad + eps)).into_dyn();;
    // 将裁剪后的v分量归一化到[-1,1]范围
    let v_normalized = v_clipped.mapv(|val| val / (maxrad + eps)).into_dyn();;

    // 返回
    compute_color(&u_normalized, &v_normalized)
}

// // 光流估计类
struct NeuFlowV2 {
    session: Session,
    input_names: Vec<String>,
    output_names: Vec<String>,
    input_height: usize,
    input_width: usize,
}

impl NeuFlowV2 {
    // 构造函数：从指定路径加载ONNX模型并初始化NeuFlowV2结构体
    fn new(path: &str) -> Result<Self> {
        // 检查模型文件是否存在及有效性（假设check_model是自定义函数）
        check_model(path)?;

        // 使用ort库创建推理会话：通过SessionBuilder加载ONNX模型文件
        // ort库是ONNX Runtime的Rust封装，支持高性能推理
        let session = Session::builder()?.commit_from_file(path)?;

        // 直接指定输入节点名称（根据提示）
        let input_names = vec![
            "input1".to_string(),
            "input2".to_string()
        ];

        // 直接指定输出节点名称（根据提示）
        let output_names = vec![
            "output".to_string()
        ];

        // 获取输入形状（根据提示中的tensor形状），float32[1,3,432,768]
        let input_shape = [1, 3, 432, 768];
        // 432
        let input_height = input_shape[2] as usize;
        // 768
        let input_width = input_shape[3] as usize;

        // 返回初始化完成的NeuFlowV2结构体
        Ok(NeuFlowV2 {
            // ONNX推理会话
            session,
            // 输入节点名称列表
            input_names,
            // 输出节点名称列表
            output_names,
            // 模型要求的输入图像高度
            input_height,
            // 模型要求的输入图像宽度
            input_width,
        })
    }

     // 光流估计：输入前后两帧图像，输出光流估计结果
     fn estimate_flow(&mut self, img_prev: &ImageBuffer<Rgb<u8>, Vec<u8>>, img_now: &ImageBuf
        // 预处理输入图像
        let (input_prev, input_now) = self.prepare_inputs(img_prev, img_now)?;
        // 执行模型推理
        let outputs = self.inference(&input_prev, &input_now)?;
        // 处理并返回输出结果
        Ok(self.process_output(&outputs[0]))
    }

    // 预处理输入：准备前后两帧图像的输入张量
    fn prepare_inputs(&mut self, img_prev: &ImageBuffer<Rgb<u8>, Vec<u8>>, img_now: &ImageBuf
        // 获取当前帧图像的尺寸
        let img_height = img_now.height() as usize;
        let img_width = img_now.width() as usize;

        // 分别预处理前后两帧图像
        let input_prev = self.prepare_input(img_prev)?;
        let input_now = self.prepare_input(img_now)?;

        // 返回预处理后的输入张量
        Ok((input_prev, input_now))
    }

    // 准备单帧输入：将图像转换为模型需要的张量格式
    fn prepare_input(&mut self, img: &ImageBuffer<Rgb<u8>, Vec<u8>>) -> Result<ArrayD<f32>> {
        // 将图像缩放到模型要求的尺寸（使用最近邻插值）
        let dynamic_img = DynamicImage::ImageRgb8(img.clone());
        let input_img_dyn = dynamic_img.resize_exact(
            self.input_width as u32,
            self.input_height as u32,
            image::imageops::FilterType::Nearest
        );
        // 将缩放后的 DynamicImage 转换回 ImageBuffer<Rgb<u8>, Vec<u8>>
        let input_img: ImageBuffer<Rgb<u8>, Vec<u8>> = input_img_dyn.clone().into_rgb8();

        // 创建4通道的零张量（1, 通道, 高度, 宽度）
        let mut input_array = Array::zeros((1, 3, self.input_height, self.input_width));

        // 遍历图像像素，归一化并填充到张量中
        for (x, y, pixel) in input_img.enumerate_pixels() {
            // R通道归一化
            input_array[[0, 0, y as usize, x as usize]] = pixel[0] as f32 / 255.0;
            // G通道归一化
            input_array[[0, 1, y as usize, x as usize]] = pixel[1] as f32 / 255.0;
            // B通道归一化
            input_array[[0, 2, y as usize, x as usize]] = pixel[2] as f32 / 255.0;
        }

        // 将固定维度的数组转换为动态维度张量
        let input_tensor = input_array.into_dyn();
        Ok(input_tensor)
    }

    // 执行推理：输入前后两帧的张量，返回输出结果
    fn inference(&mut self, input_prev: &ArrayD<f32>, input_now: &ArrayD<f32>) -> Result<Vec<
        // 记录推理开始时间
        let start = Instant::now();
        // 将输入张量转换为ORT支持的Value类型
        let input_tensor_prev = ort::value::Value::from_array(input_prev.clone().into_dyn())?
        let input_tensor_now = ort::value::Value::from_array(input_now.clone().into_dyn())?;

        // 执行模型推理（输入两个张量）
        let outputs = self.session.run(inputs![input_tensor_prev, input_tensor_now])?;

        // 计算并打印推理耗时
        let elapsed = start.elapsed();
        println!("推理时间: {:.2} ms", elapsed.as_secs_f64() * 1000.0);

        // 提取所有输出张量
        use ort::value::{ Value, TensorValueType };
        let mut result: Vec<Tensor<f32>> = Vec::new();
        for output_name in &self.output_names {
            let (shape, data) = outputs[output_name.clone()].try_extract_tensor::<f32>()?;

            // 将 ort::Shape 转换为 ndarray 兼容的维度（动态维度）
            let nd_shape: Vec<usize> = shape.iter().map(|&d| d as usize).collect();
            let array = Array::from_shape_vec(IxDyn(&nd_shape), data.to_vec())?;

            let output = Value::from_array(array)?;
            result.push(output);
        }

        // 返回值
        Ok(result)
    }

    // 处理输出：将输出张量转换为合适的格式
    fn process_output(&mut self, output: &Tensor<f32>) -> ArrayD<f32> {
        // 解构输出张量（形状和数据）
        let (shape, data) = output.try_extract_tensor::<f32>().unwrap();
        // 根据形状创建4维数组（1, 通道, 高度, 宽度）
        let array = Array::from_shape_vec(
            Dim::<[usize; 4]>::new([1 as usize, shape[1] as usize, shape[2] as usize, shape[3
            data.to_vec()
        ).unwrap();
        // 转换为动态维度张量
        let array_dyn = array.into_dyn();
        let flow = array_dyn.to_shape(
            Dim::<[usize; 4]>::new([1 as usize, shape[1] as usize, shape[2] as usize, shape[3
        ).unwrap();
        // 映射所有值（此处未做实际转换，保留原值）
        let resized_flow = flow.mapv(|val| val);
        let resized_flow_dyn = resized_flow.into_dyn();
        // 返回
        resized_flow_dyn
    }

}

fn main() -> Result<()> {
    // 模型文件路径
    let model_path = "../../assets/ailia-models/neuflow_v2/neuflow_sintel.onnx";
    println!("模型文件路径{}", model_path);

    // 初始化模型
    let mut estimator = NeuFlowV2::new(model_path)?;

    // 加载第一张图片
    let img1 = image::open("./assets/frame_0016.png")?.to_rgb8();
    // 加载第二张图片
    let img2 = image::open("./assets/frame_0025.png")?.to_rgb8();

    // 估计光流
    let flow = estimator.estimate_flow(&img1, &img2)?;

    // 绘制光流图
    let flow_img = flow_to_image(&flow);

    // 创建结果文件夹
    std::fs::create_dir_all(Path::new("./result"))?;

    // 保存结果图片
    flow_img.save("./result/ort_optics.png")?;
    println!("光流预测结果已保存到 ./result/ort_optics.png");

    Ok(())
}
```

效果

图片1	图片2
	

Python结果	Rust结果
	

分类： 折腾笔记

标签： Rust ， SLAM

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 折腾笔记[23]-使用rust进行图像分割

posted @ 2025-04-14 05:20 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】凌霞软件回馈社区，携手博客园推出1Panel与Halo联合会员
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

相关博文：
- 折腾笔记[21]-使用rust推理onnx模型
- 折腾笔记[23]-使用rust进行图像分割
- 2024-08-22-LET-Net 中英对照
- 2024-08-28-RAFT-中英对照
- RAFT光流估计

阅读排行：
- 精选 4 款免费且实用的数据库管理工具，程序员必备！
- Cursor：一个让程序员“失业”的AI代码搭子
- 【杭电多校比赛记录】2025“钉耙编程”中国大学生算法设计春季联赛（6）
- 重生之我是操作系统(七)----内存管理(上)
- .NET 阻止Windows关机以及阻止失败的一些原因

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes